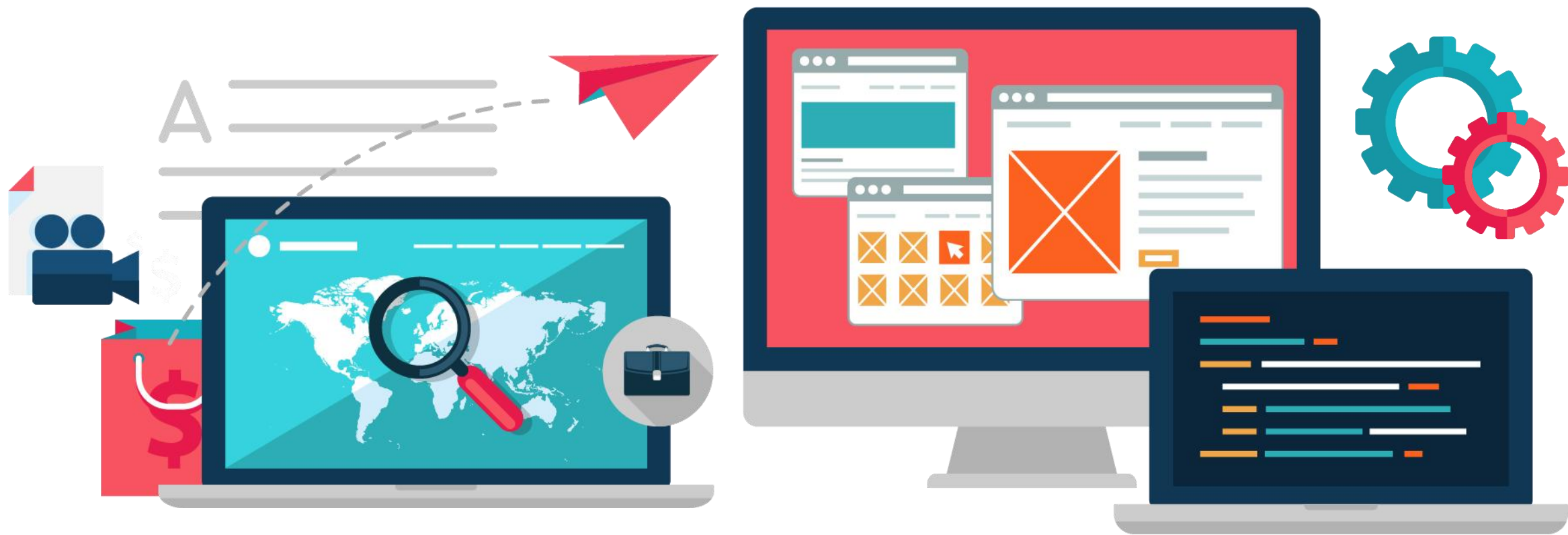


面向对象程序设计

Object Oriented Programming





拷贝和赋值



01

对象的拷贝

02

拷贝构造函数

03

缺省拷贝构造函数

04

浅拷贝

05

使用自定义拷贝构造函数

06

对象的赋值

07

自定义赋值函数





总结T类对象的创建(实例化)形式:

1. 无参创建, 显式调用默认构造函数(explicit允许)

`T obj1; T obj2{}; // 对象定义, 无括号默认初始化, 带括号值初始化`

`new T; new T{}; new T(); // 动态创建无名对象, 初始化同上`

`T{}; // 创建临时无名对象, 值初始化, 显式类型转换, 常用于函数传参/返回`

`// 成员初始化列表中的mt()或mt{}是成员初始化器, 对象成员T mt定义的一部分`

`T arr1[2]; T arr2[2]{}; T arr3[5] = {T{}, T{}}; // 对象数组定义, 初始化同上`

`new T[2]; new T[2](); new T[2]{}; // 动态创建对象数组, 初始化同上`

2. 有参创建, 显式调用带参构造函数

`T obj3(10); T obj4{8,9}; // 对象定义, 直接初始化, 直接列表初始化`

`new T(10); new T{8,9}; // 动态创建无名对象, 初始化同上`

`T{8,9}; // 创建临时无名对象, 直接列表初始化, 显式类型转换, 常用于函数传参/返回`

`// 成员初始化列表中的mt(10)或mt{8,9}是成员初始化器, 对象成员定义的一部分`

`T arr1[2]{ T(10), T(8,9) }; T arr3[5]{ T{10}, T{8,9} }; // 对象数组定义, 初始化同上`

`new T[2]{ T(10), T(8,9) }; new T[5]{ T{10}, T{8,9} } // 动态创建对象数组, 初始化同上`



下面给出第3类对象创建(实例化)形式:

3. 有1个T类对象参数

`T obj5(obj1); T obj6{ T} ;` // 对象定义, 直接初始化, 直接列表初始化, 显式调用拷贝构造函数

`T obj7=obj2;` // 对象定义, 复制初始化(左值), 隐式调用拷贝构造函数

`T obj8=T{10};` // 对象定义, 复制初始化(纯右值), C++17拷贝省略, 只调用带参构造

`new T(obj3); new T{ T{8,9} };` // 动态创建无名对象, 直接初始化, 直接列表初始化

// 成员初始化列表中的`mt(obj1)`或`mt{obj1}`是成员初始化器, 对象成员`T mt`定义的一部分

◆ 第3类对象创建通常称为对象拷贝(构造): 用一个现有的同类对象构造一个新对象, 新对象是现有对象的副本(内容相同, 但二者相互独立)

◆ 通过对象拷贝进行对象定义时, 对象名前必须有完整类型`T`, 否则是对象赋值:

`T obj5(obj1);` // 对象拷贝

`obj3 = obj1;` // 对象赋值

◆ 对象拷贝时系统自动调用拷贝构造函数

拷贝构造函数:

- 是一种特殊的构造函数，语法特性相同，第一个参数是自身类类型的引用，可以有其他默认参数
- 常见: `T(const T& rhs);`
- 不能传值，避免无限递归
- 通常使用`const`型参数

C++11中新增移动构造函数:

`T(T && rhs);`

访问控制:

- `public`
- `protected`
- `private`

```
class My {
public:
    MyClass( int ) { } //自定义带参构造函数
    My(const My & rhs):val(rhs.val) { }
private: int val; //自定义拷贝构造函数
}; // 对象初始化规则与之前构造函数相同
```

拷贝构造函数也可以是T(T & rhs)形式:

```
class My {
public:
    My( My & rhs) {
        pValue = rhs.pValue;
        rhs.pValue = nullptr;
    }
private: int * pValue;
}; // 所有权转移的拷贝构造, C++98/03时
// 代遗留模式, 现代C++不常见, 应避免
```

◆显式调用(explicit允许):

```
A a1;  
A a2(a1);
```

◆隐式调用(explicit禁止):

```
A a3=a1;  
  
void Function( A aA) { }  
A obj; Function(obj);  
  
A Function( ) {  
    A aA; return aA;  
}
```

返回值优化(URVO和NRVO)

是编译器对于函数返回值的一种优化技术, 旨在消除临时对象的创建, 提高程序运行效率

显式调用例:

```
class A {  
public:  
    A(int n);  
    A(const A& rhs);  
    ...  
};  
  
class B {  
public:  
    B(A& aA):a(aA){ } //调用拷贝构造函数  
    B(int n):a(n) { } //调用带参构造函数  
private:  
    A a;  
};
```

最基本的情况(无优化): 2次拷贝构造、1次有参构造

```
People func() {  
    People temp_a("temp name");  
    return temp_a;  
}  
  
int main() {  
    People a = func();  
    return 0;  
}
```

- Step 1 : 开辟 a 对象数据区
- Step 2 : 调用函数 func
- Step 3 : 开辟对象 temp_a 数据区
- Step 4 : 调用 temp_a 对象的构造函数
- Step 5 : 使用 temp_a 调用临时匿名变量的拷贝构造函数
- Step 6 : 销毁 temp_a 对象
- Step 7 : 使用临时匿名变量调用 a 的拷贝构造函数
- Step 8 : 销毁临时匿名变量
- Step 9 : 销毁 a 对象

优化一：1次拷贝，1次有参构造（VC++可以看到一次拷贝的现象）

```
People func() {  
    People temp_a("temp name");  
    return temp_a;  
}  
  
int main() {  
    People a = func();  
    return 0;  
}
```

Step 1：开辟 a 对象数据区

Step 2：调用函数 func

Step 3：开辟对象 temp_a 数据区

Step 4：调用 temp_a 对象的构造函数

Step 5：使用 temp_a 调用 a 的拷贝构造函数

Step 6：销毁 temp_a 对象

Step 7：销毁 a 对象

优化二：0次拷贝，1次有参构造

temp_a就是a，a就是temp_a，只有一次有参构造，有参构造构造的既是temp_a对象，也是a对象。

gnu c++ 去掉 -fno-elide-constructors 编译选项即可

自定义拷贝构造函数:

缺省的拷贝构造函数:

- 没有显式声明拷贝构造函数时, 由编译器隐式声明
- 隐式声明的是public、inline的
- 拷贝方式是“浅”拷贝;
- “浅”拷贝有时不能满足要求;

“浅”拷贝: 隐式声明的拷贝构造函数, 对每个非静态数据成员, 进行位拷贝(对内置类型)或调用其拷贝构造函数(对类类型), 以源对象的对应成员进行初始化。

显式调用例:

```
class A {  
public:  
    A(int n);  
    A(const A& rhs); //编译器可隐式生成  
    ...  
};  
class B {  
public:  
    B(A& aA):a(aA) { } //调拷贝构造函数  
    B(int n):a(n) { } //调构造函数  
    // 未显式声明时编译器一般隐式生成如下格式  
    // B(const B& rhs):a(rhs.a) { }  
private:  
    A a;  
};
```

- “浅” 拷贝的本质
- 数据成员的“浅” 拷贝
 - 静态数据成员
 - 内置类型数据
 - 指针数据成员
 - 引用数据成员
 - 对象数据成员：自动调用对象所属类的拷贝构造函数

浅拷贝说明：

```
class AA { /* 略 */ };
```

```
class My {  
public:  
    My (AA & a ) : mRefAA(a)    { }  
private:  
    int    mVal;  
    AA *   mpAA;  
    AA&    mRefAA;  
    AA     mAA;  
};
```

浅拷贝说明:

```
class AA { /* 略 */ };
```

```
class My {
```

```
public:
```

```
    My (AA & a )
```

```
        : mRefAA(a) { }
```

```
private:
```

```
    int mVal;
```

```
    AA * mpAA;
```

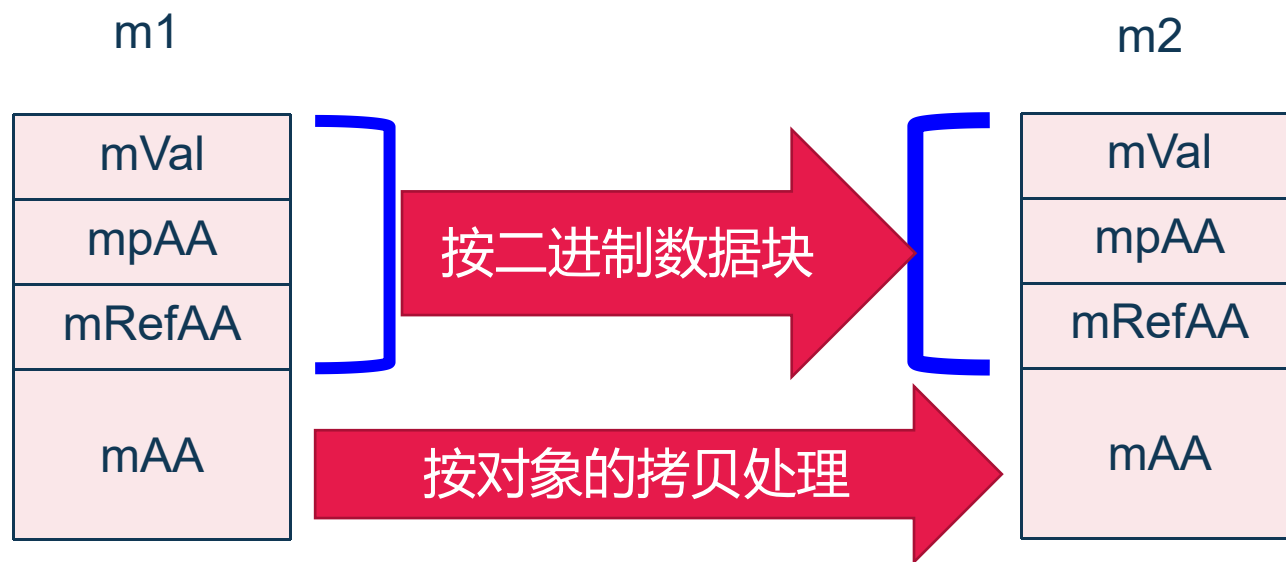
```
    AA& mRefAA;
```

```
    AA mAA;
```

```
};
```

```
My m1;
```

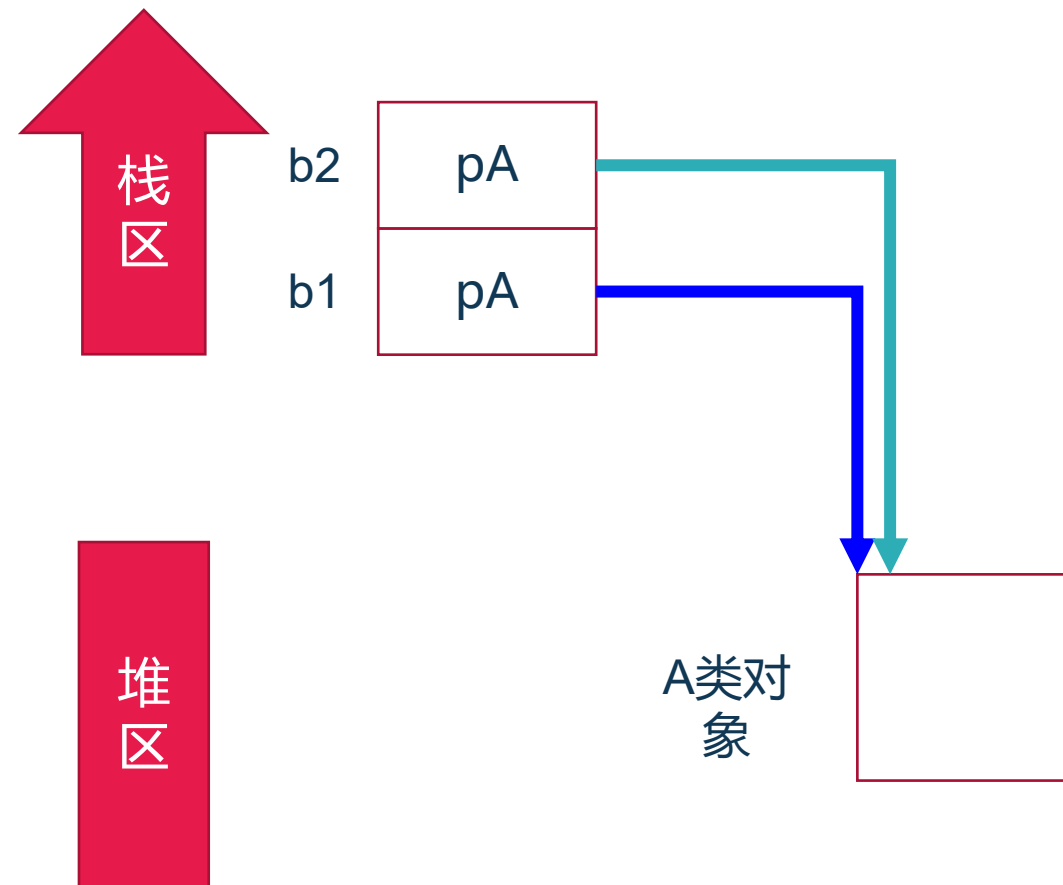
```
My m2(m1); //拷贝
```



浅拷贝不足(例)

```
class A { /* 略 */ };  
class B {  
public:  
    B() { pA = new A; }  
    ~B() { delete pA; }  
private:  
    A * pA;  
};
```

```
int main( ) {  
    B b1;  
    B b2(b1);  
    return 0;  
}
```



- ✓ 按程序员的目的实现拷贝
- ✓ 深拷贝只能通过自定义拷贝构造函数实现

```
int main( ) {  
    B  b1;  
    B  b2(b1);  
    return 0;  
}
```

```
//例:  
class B {  
public:  B( )    {  pA = new A;  }  
        ~B( )   {  delete pA;   }  
        B(const B& b) :rA(b.rA),aA(b.aA)  
                        {  pA= new A(*b.pA);  }  
        //或者  
        B(const B& b)  
            :pA(new A(*b.pA)), rA(b.rA),aA(b.aA)  
            {  }  
private:  
    A *  pA;  
    A &  rA;  
    A    aA;  
};
```



需要自定义拷贝构造函数：

- a. **深拷贝时；** 当类管理动态资源（如堆内存、文件句柄）时，默认的浅拷贝会导致多个对象共享同一资源，析构时重复释放。需自定义拷贝构造实现深拷贝（分配新资源并复制内容）。
- b. **禁止拷贝；** 某些类不应被拷贝（如单例、管理唯一资源的类）。可通过将拷贝构造函数声明为 `delete` 或 `private`（C++11前）来禁止。
- c. **防止按值传递对象；** 按值传递会调用拷贝构造。若想强制只能传递指针或引用，可将拷贝构造设为私有或删除。通常与禁止拷贝相同。
- d. **程序员的其它特殊目的，如引用计数、所有权转移等，** 现代 C++ 中，所有权转移宜用移动语义，引用计数用 `shared_ptr`，深拷贝仍需手动实现。

禁止拷贝和防止值传递对象(例)

```
class My {  
private:  
    My(const My& m) { } //允许类内拷贝  
  
    //或者, 采用更严格的, 避免类内拷贝  
    //My(const My& m); //且没有类外实现  
  
    //或者C++11后  
    //My(const My&) = delete;  
    //函数声明时被标记为已删除, 且没有可调  
    //用的定义, 使用会导致编译错误  
};
```

```
My obj;  
  
My obj2=obj;  
  
My obj3(obj);  
  
void Wrong03(My m)  
{ /*略*/ }  
  
My Wrong04( )  
{ /*略*/ }
```

= delete可以用于任何函数声明, 包括且不限于普通成员函数、特殊成员函数、运算符重载函数、非成员函数、模板函数等。必须确保任何函数的=delete标记都只出现在它的第一次声明中



比较 有实现和无实现的差异(例)

```
class My {  
private:  
    My(const My& m) { /*略*/ }  
private:  
    void f() {  
        My m1;  
        My m2(m1); //合法  
    }  
};  
int main() {  
    My m1;  
    My m2(m1); //编译错误  
    return 0;  
}
```

```
class My {  
public:  
    My(const My& m); //没有类外实现  
private:  
    void f() {  
        My m1;  
        My m2(m1); //链接错误  
    }  
};  
int main() {  
    My m1;  
    My m2(m1); //链接错误  
    return 0;  
}
```

结论：若要完全禁止拷贝，最好自定义一个private且没有实现的拷贝构造函数，在C++11以后，也可以使用 `My(const My& m)=delete;`



含义：使用**已存在的同类对象**修改当前对象，称为对象的**拷贝赋值**。

比较复制(拷贝)与拷贝赋值

- ✓ 复制：从无到有创建一个新对象，新对象是源对象的副本
- ✓ 拷贝赋值：当前对象已经构造完成，只是修改

```
int main() {  
    A  a1;      // 默认构造  
    A  a2(a1);  // 拷贝构造(复制)  
    A  a3=a2;   // 拷贝构造(复制)  
    a3 = a1;    // 拷贝赋值(a3已存在,  
                // 需要拷贝赋值函数)  
    int num = 99;  
    a2 = 99;    // 普通赋值(需要赋值  
                // 运算符重载函数)  
    return 0;  
}
```

对象拷贝赋值时系统自动调用**拷贝赋值函数**，是赋值运算符重载函数的一种特殊形式，其参数必须是本类的类型(const T& 或 T& 或 T等)

- 没有显式给出拷贝赋值函数时，编译器隐式声明
- 隐式声明是public、inline的
- 采用浅赋值
 - 含义类似浅拷贝
 - 对象成员的赋值
 - 非静态引用或const成员不能赋值
 - 浅赋值的不足

```
int main() {  
    AA a1;  AA a2;  
    My m1(a1); My m2(a2);  
    //类中有非静态引用或const数据成员时  
    //提示:尝试使用已删除的缺省赋值函数  
    m1 = m2;  
}
```

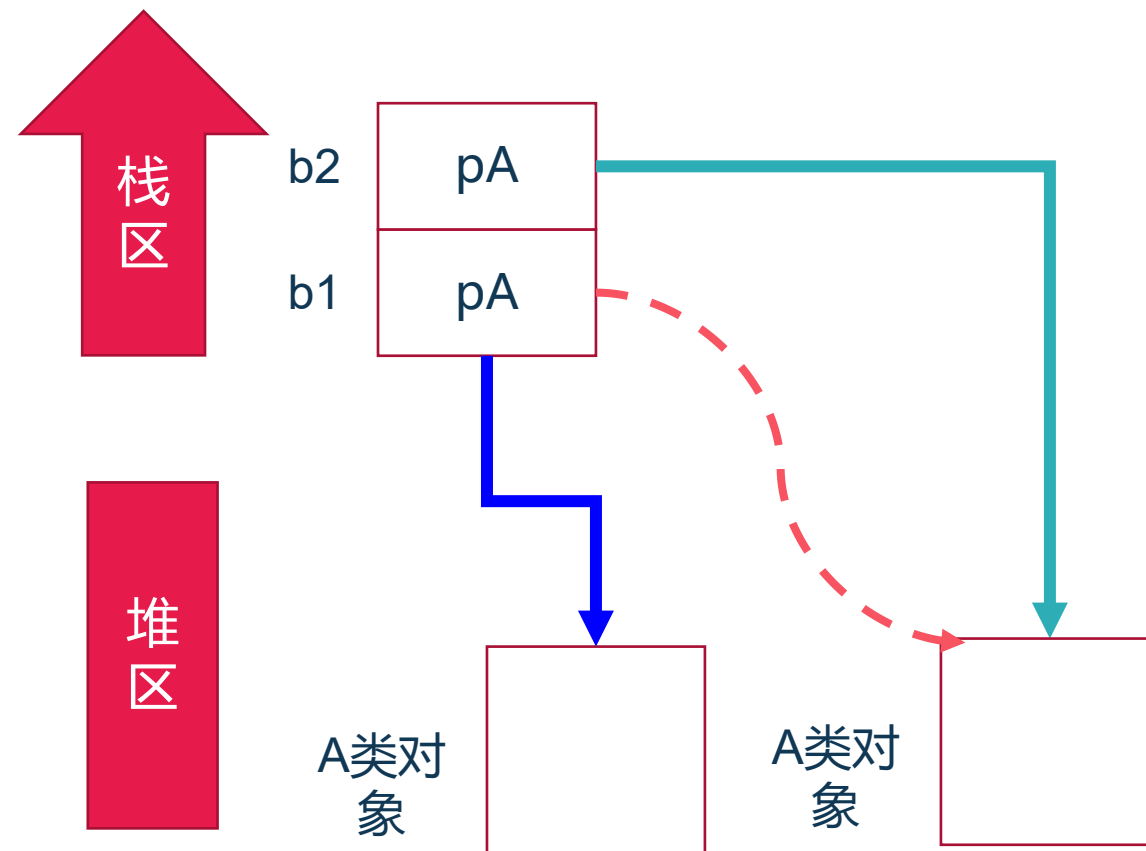
浅赋值说明:

```
class AA { /* 略 */ };  
  
class My {  
public:  
    My(AA& ra) : mRefAA(ra) { }  
private:  
    int mVal;  
    AA * mpAA;  
    AA& mRefAA; //const int c=10;  
    AA mAA;  
};
```

浅赋值的不足(例)

```
class A { //略 };  
  
class B {  
public:  
    B() { pA = new A; }  
    ~B() { delete pA; }  
private:  
    A * pA;  
}
```

```
int main() {  
    B b1, b2;  
    b1 = b2;  
    return 0;  
}
```





- 格式: `T& operator= ( [const] T & rhs );`
- 非构造函数, 无成员初始化列表
- 返回 `*this` 的必要;
- 实现中检查自我赋值
 - 效率
 - `a = a` 的情况

```
class My {  
    public:  
        My & operator=(const My& rhs ) {  
            ...  
            return *this;  
        }  
};
```

- `int a=1,b=2,c=3;`
- `a=(b=c);`
`a=b=c;`
`(a=b)=c;`



- `A a,b,c;`
- `a=(b=c);`
`a=b=c;`
`(a=b)=c;`

反例1:

```
class My {  
    public:  
    void operator=(const My& rhs ) ;  
};  
  
My m1,m2,m3;  
m1=m2=m3;  
(m1=m2)=m3;
```

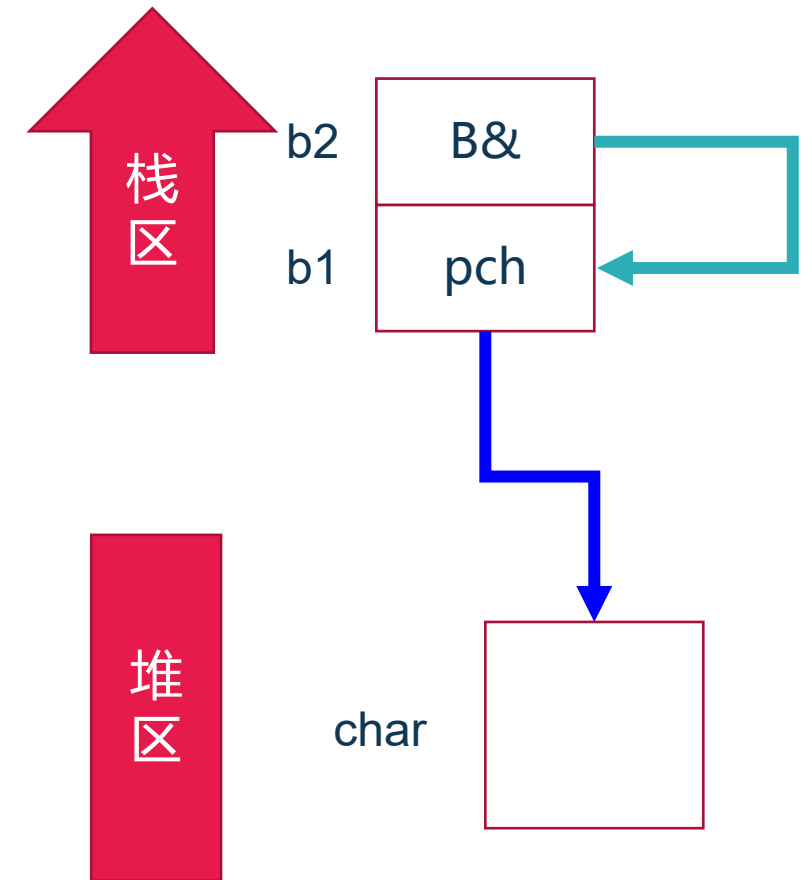
反例2:

```
class My {  
    public:  
    My operator=(const My& rhs ) ;  
};  
  
My m1,m2,m3;  
m1=m2=m3;  
(m1=m2)=m3;
```

拷贝赋值函数实现中的自我赋值判定

```
class B {  
public:  
    B() { pch = new char; }  
    ~B() { delete pch; }  
    B& operator=(const B& rhs) {  
        delete pch;  
        pch = new char(*rhs.pch);  
        return *this;  
    }  
private:  
    char * pch ;  
};
```

```
int main( ) {  
    B b1; B & b2= b1;  
    b1 = b1; b1 = b2;  
    return 0;  
}
```





赋值函数的一般实现

```
class B {  
public:  
    B( ) { pch = new char; }  
    ~B( ) { delete pch; }  
    B& operator=(const B& rhs) ;  
private:  
    char * pch ;  
};
```

```
int main( ) {  
    B b1; B & b2= b1;  
    b1 = b1; b1 = b2;  
    return 0;  
}
```

```
B& B::operator=(const B& rhs)  
{  
    if ( &rhs != this ) {  
        delete pch;  
        pch = new char(*rhs.pch);  
    }  
    return *this;  
}
```




本章结束